

Automating Excel Tasks with Generative AI: Supporting Innovation in Mass Transfer Separations Education

Moisés García-Morales*

Cite This: *J. Chem. Educ.* 2026, 103, 1064–1070

Read Online

ACCESS |



Metrics & More



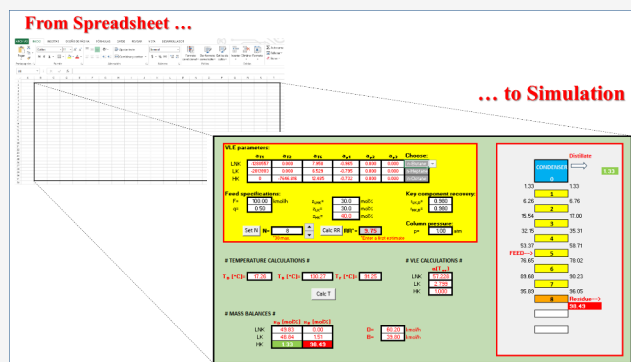
Article Recommendations



Supporting Information

ABSTRACT: VBA (Visual Basic for Applications) macro creation assisted by AI (Artificial Intelligence) can enhance the teaching of Mass Transfer Separations. The code can be iteratively refined until the desired functionality is achieved. This is not only intended to improve computational efficiency and accuracy but also to boost student engagement. To showcase the full potential of VBA scripting within spreadsheets, a simulator with an appealing interface was designed using Excel, such that optimal and limiting operating conditions were identified in an equilibrium-staged distillation column. Moreover, to underscore the pedagogical value of the approach at this exploratory stage of evaluating its educational scope, a much simpler classroom exercise was used to illustrate the hands-on integration of GenAI (Generative AI) into the learning process.

KEYWORDS: Upper-Division Undergraduate, Chemical Engineering, Computer-Based Learning, Mathematics/Symbolic Mathematics, Separation Science



1. INTRODUCTION

Simulation tools, like Aspen Plus or Aspen Hysys, are crucial for modeling and optimizing separation operations.^{1,2} For this reason, process simulators have been extensively used in Higher Education.^{3–6} However, their complexity and cost can occasionally hinder accessibility for students and smaller academic initiatives. Moreover, they have the disadvantage that the equations and the solution strategy are not always perceived by the students.^{7,8} Hence, the use of simpler software packages that require model implementation may help mitigate the so-called “black-box” effect.⁹

Microsoft Excel has emerged as a practical alternative for process simulations, especially in educational settings. Research shows that Excel can be used as a problem solving and optimization tool for many Chemical Engineering core courses.^{10–14} The use of simple or nested logical functions in Excel enables users to create automated decision-making processes that adapt to various operational scenarios.¹⁵

Beyond the application of simple built-in functions, Excel's macros can automate iterative tasks, streamlining the simulation process.¹⁶ This not only enhances efficiency but also reduces the likelihood of errors in complex scenarios.^{10,17} Moreover, the practical use of MS Excel and VBA (Visual Basic for Applications) programming, widely recognized by industry as essential tools, is an often overlooked but critical gap in engineering curricula.¹⁸

In this context, and despite ongoing critiques of AI (Artificial Intelligence) in Higher Education, its application in writing codes may demonstrate its potential as a powerful tool to support student learning.¹⁹ AI can assist students in creating and using macros, even without prior knowledge of VBA programming, thereby challenging negative perceptions of AI's role in academia.²⁰ In that sense, AI can significantly improve the programming of VBA macros in Excel for automating, for example, distillation calculations, transforming concerns about AI in education into opportunities.^{20,21}

Commercial simulators can sometimes bypass fundamental learning by focusing more on input-output data rather than the mechanics of the calculations.^{7,22} In contrast, the proposed Excel model prioritizes educational value, even in distance learning settings.²³ It encourages students to engage with underlying calculations, use logical functions to replicate operational decisions, and automate routines with AI tools.²⁴ This hands-on approach deepens their understanding of mass transfer concepts and prepares them for practical problem-solving. Excel requires students to manually input equations, design

Received: October 20, 2025

Revised: December 16, 2025

Accepted: December 30, 2025

Published: January 7, 2026



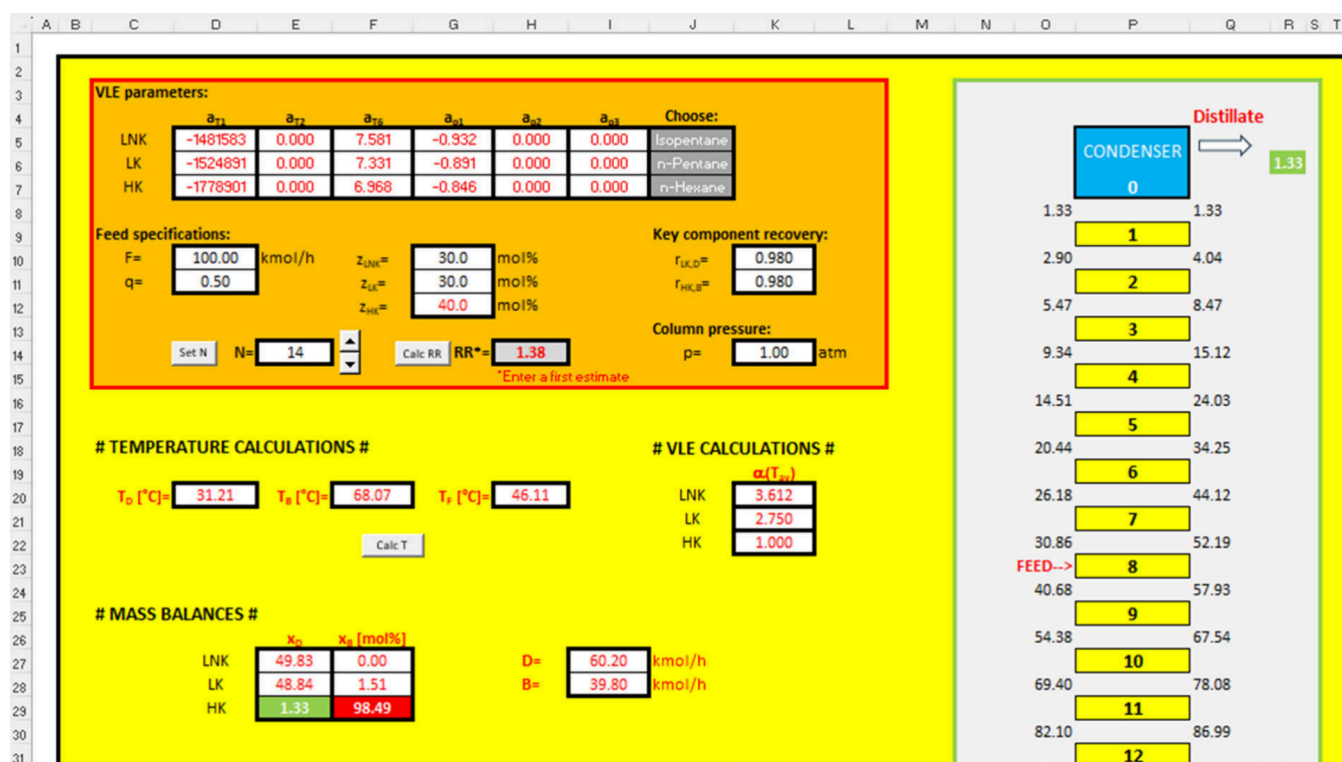


Figure 1. Top part of the Excel-based software interface, including VLE parameters, operational specifications, VLE calculations, mass balances and column overhead.

calculation steps, and set up iterative processes, providing a more comprehensive understanding of how simulators function behind the scenes.²⁵

Through this work, we aim to show how programming a simulation in Excel, supported by AI-driven macro development, might lead to more meaningful and inclusive learning experiences: meaningful, because AI provides detailed explanations of generated code that enable the users to learn from it and apply it to future projects; inclusive, because the method is accessible not only to users with some previous programming experience but also to those with limited knowledge.

2. SHOWCASING THE POWER AND VERSATILITY OF VBA MACROS

Understanding multistage distillation is of great value to practicing engineers in the process industry. Hence, a home-made simulator for fractional distillation is herein used to demonstrate the full potential of programming applications in Chemical Engineering (Excel file included as Supporting Information, SI_5). In contrast to complex matrix methods^{26,27} used in commercial simulators, a step-by-step method can be easily implemented and applied to multicomponent mixtures as long as two main constraints are met: (i) either heavy nonkey (HNK) or light nonkey (LNK) components are not present; (ii) there are no components distributed between the heavy key (HK) and light key (LK) components.^{25,28} For a comprehensive overview of the procedure and the complete set of equations (eqs 1 to 14), please refer to the Supporting Information (SI_1a).

The strategy aims to enhance classical methodologies in equilibrium-staged separation training by incorporating a transversal approach focused on developing ICT (Information and Communication Technology) competencies.¹⁵ Specifically,

this initiative seeks to foster digital literacy among Higher Education students using spreadsheets in engineering calculations.²⁹

Figure 1 presents the top part of the software interface. As seen, the interface is enclosed within a window with a yellow background. Vapor–liquid equilibrium (VLE) parameter values, feed specifications, key component recoveries and number of equilibrium stages are all encompassed within an orange box at the top. The number of equilibrium stages can be adjusted between 1 and 30 using a spin button. The reflux ratio is set as a manipulated (or adjusting) variable. A first estimate (initial value) is needed before the simulation can be run. Components can be selected from a dropdown list. Once selected, their VLE parameter values are automatically loaded into the corresponding cells.

External column mass balances enable mole fractions and flow rates in distillate and bottoms to be determined, according to the specified key component recoveries and assuming that the LNK bottoms mole fraction is zero (when HNKs are present, $x_{\text{HNK},D} = 0$ is assumed). This can be done by simply writing the explicit algebraic expressions (eqs 11 to 14 shown in SI_1a) in the corresponding cells. Table 1 illustrates cells E27 to E29 and I27, using the distillate as an example. The results are shown in Figure 1. HK mol % in the distillate and bottoms are highlighted to facilitate clearer identification of the target compositions.

Unlike the mass balances, temperature calculations are not straightforward. Equation 5a, 5b and 4 need to be solved for T_D , T_B and T_F , respectively. The “GoalSeek” command was used for that purpose. Such a command is located in the toolbar and can be accessed by following the sequence: Data > What-If Analysis > GoalSeek. A target cell which contains a formula is forced to achieve a specified value (Goal) by modifying the value in a changing cell. An initial estimate in the changing cell is

Table 1. Code in Selected Cells and Related Equations in SI_1a

Cell	Code	Related equations in SI_1a
E27	D11*G10/I27	Equation 11
E28	D11*K10*G11/I27	Equation 12
E29	D11*(1-K11)*G12/I27	Equation 13
I27	D11*(G10+K10*G11+(1-K11)*G12)/100	Equation 14
Q10	IF(P9 = "";"", (O8/K\$22)/(N8/K\$20+N9/K\$21+O8/K\$22)*100)	Equation 2
O10	IF(P11 = "";"", IF(R11/Q10 > L\$42; E\$45*Q10+F\$45; G\$45*Q10+H\$45))	Equation 10; eq 7a, 7b
O11	IF(R13 = "";"", IF(AND(R11/Q10 > L\$42; R13/Q12 < L\$42); "FEED->" ; ""))	Equation 10

mandatory. However, if one or more of the variables involved in the above-mentioned formula changes the solutions do not change. This issue is addressed by using a macro, which can be easily recorded from the Developer tab > Record Macro, and then assigned to a button to facilitate its execution whenever required.

For the calculation of T_D , a macro was recorded and named “calcTD”. Its code is shown in Table 2. The “GoalSeek” command is remarkably straightforward to automate: Range(target_cell).GoalSeek Goal:=target_value, ChangingCell:=Range(manipulated_cell). The target cell, W7, represents the sum on the left-hand side of eq 6a (not shown to enhance the interface’s appearance and improve the user experience). The Goal corresponds to the value on the right-hand side of eq 6a, which is 100% (or 1, in molar fraction), while the manipulated cell, D20, represents the unknown, i.e., T_D . For the sake of clarity, the process is explained in a video which is provided as Supporting Information (SI_2).

Instead of using three different buttons, students are taught to find the three temperatures, i.e., T_D , T_B and T_F , simultaneously. Based on prior experience, students can link the instruction to its outcome, enhancing their ability to apply learned concepts to similar situations. This approach aligns with inductive learning theory, where students generalize principles from specific instances.³⁰ Within the same macro, the “GoalSeek” command can be used multiple times. The process is quite intuitive: students simply need to reuse the same instruction, replacing the previous target_cell, target_value, and manipulated_cell with the new ones. The new macro “calcT” was assigned to the “Calc T” button. For the sake of clarity, the process is explained in a video which is provided as Supporting Information (SI_3).

Further information on the software interface is provided as Supporting Information (SI_1b).

The column design was started with the use of simple built-in functions. Especial emphasis was placed on the logical “IF” statement. The spreadsheet was programmed for a maximum of 30 arbitrarily chosen equilibrium stages. As LNKs are present,

the stepping off process needs to start from the top (when HNKs are present, it starts from the bottom). LNKs are assumed to be entirely collected at the distillate (if present, HNKs are entirely collected at the bottoms). Starting from the top, HK mol % in liquid exiting the ‘n = 1’ stage is calculated as in cell Q10 (Table 1), according to eq 2. The “IF” statement used is intended to apply the formula only if the ‘n = 1’ stage is active (colored and numbered), as shown in Figure 1. This process is repeated at every stage. When a stage is inactive (no color, no number), no calculations are performed and the cell appears empty, as shown in SI_1b below stage 14. Values for LK and LNK remain always hidden, for the sake of a more visually appealing and user-friendly interface. The use of special paste, relative cells, and absolute cells in MS Excel facilitates the task. Moreover, nested “IF” functions in cell O10 (Table 1) allowed for multiple criteria to be satisfied. The first “IF” statement, again, is intended to apply the formula only if the ‘n = 2’ stage is active. If this is the case, the ROL is used if (second condition) the optimal feed location has not yet been achieved. Otherwise, the SOL will be used. Moreover, the instructions in cell O11 aims to make visible the optimal feed location, which is indicated by the text “FEED->”. Optimal feed location, n_F , fulfils the following requirement: $(x_{LK}/x_{HK})_{n_F-1} > \text{ratio}_{LK,HK} > (x_{LK}/x_{HK})_{n_F}$. That is why the instruction requires the use of the “AND” function, which checks if both conditions are true. Again, the process is repeated at every stage. If ‘n+1’ is inactive, it means that ‘n’ is the reboiler, which is highlighted in a different color (as shown in SI_1b). Note that the total condenser is set to $n = 0$ as it does not function as an equilibrium stage.

The subsequent step involved defining the number of equilibrium stages, with a maximum limit of 30. Active stages are sequentially numbered from the top down, while inactive stages remain blank. Completing this task necessitates the development of a VBA macro, for which AI tools can provide valuable assistance.³¹ Copilot (<https://copilot.microsoft.com/>), developed by Microsoft, offers a free and effective solution.³² Chat GPT (<https://chatgpt.com/>) or Gemini (<https://gemini.google.com/app>), for example, would be equally valid.³² The key to success lies in providing the AI assistant with a detailed and precise description of the desired functionality and iteratively refining the output through multiple attempts.³² The required macro was aimed at automating the process of filling and formatting cells in an Excel worksheet based on a specified number of stages. The whole sequence of different attempts that improved macro performance, including the outputs returned by Copilot, is provided as Supporting Information (SI_1c). The “N” macro, refined after several attempts, can be easily implemented by accessing the Developer tab in Excel, opening the Visual Basic Editor, inserting a new module, and pasting the provided code. The macro was assigned to the “Set N” button, facilitating its usage. A video tutorial

Table 2. Recorded Excel Macros for Temperature Calculations

Sub calcTD() Range("W7").GoalSeek Goal:=100, ChangingCell:=Range("D20") End Sub
Sub calcT() Range("W7").GoalSeek Goal:=100, ChangingCell:=Range("D20") Range("X7").GoalSeek Goal:=100, ChangingCell:=Range("F20") Range("Y7").GoalSeek Goal:=0, ChangingCell:=Range("H20") End Sub

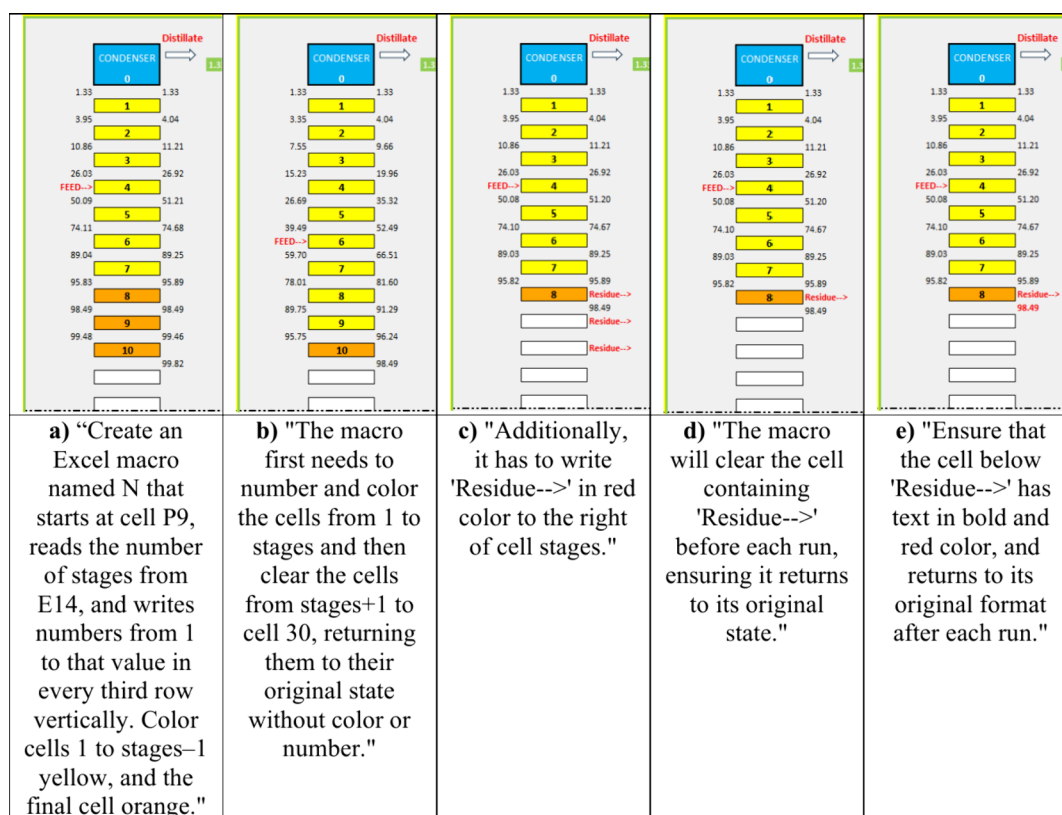


Figure 2. Evolution of the Excel-based software performance upon different Copilot rounds: (a) first; (b) second; (c) third; (d) fourth; (e) fifth.

Table 3. Final "RR" Excel Macro Using Copilot

```
Sub RR()
    Dim numberStages As Double
    Dim destinationCell As Range
    Dim startingPoint As Range

    ' Retrieve the number of stages
    numberStages = Range("E14").Value

    ' Set the starting point for the stages
    Set startingPoint = Range("Q10")

    ' Determine the destination cell based on the number of stages
    Set destinationCell = startingPoint.Offset((numberStages - 1) * 2, 0)

    ' Perform Goal Seek to adjust the reflux ratio to achieve the target value
    destinationCell.GoalSeek Goal:= Range("F29").Value, ChangingCell:= Range("H14")
End Sub
```

detailed the implementation process is included as Supporting Information (SI_4).

As shown in Figure 2, the first attempt failed because it did not refresh the column with each run (a), causing the effects of subsequent runs to overlap. The second attempt was successful, but there was still room for improvement (b). For instance, the third round aimed to emphasize the location of the residue stream. However, the effects of multiple runs overlapped again (c). After consulting with Copilot, the code was debugged to clear the results of previous runs (d). The fifth round successfully highlighted the HK mol % in the residue stream in bold red (e). After the fifth round, only minor details were added to the software to enhance its interface, as shown in Figure 1.

Similarly, a macro to calculate the reflux ratio, according to the algorithm in SI_1a, was developed using Copilot iteratively

(Table 3), and assigned to the "Calc RR" button. When the macro is executed, the reflux ratio (RR) is calculated to ensure that the residue composition in the column matches the value in the mass balance table.

This method immerses students in practical, hands-on learning environments where they can directly observe the impact of their modifications.³³ A case study corresponding to the multistage distillation of a ternary mixture is presented in Supporting Information (SI_1d). Expected learning outcomes, benefits, opportunities and limitations of this problem are reported.

It is noteworthy that in the Microsoft 365 subscription version of Excel, the "Add-ins" button can be found in the Home tab, allowing users to install a wide range of applications that integrate advanced AI technologies directly into Excel (SI_1c). However, these add-ins are not ideally suited for beginner

learners because handling VBA modules is not mandatory, thereby limiting opportunities for students to develop coding literacy and critical thinking. On the contrary, using free Copilot to generate VBA code not only offers the advantage of accessibility, but also fosters deeper engagement with programming concepts. Learners must copy the generated code into the Excel module, interpret its logic, and adapt it to their specific tasks.

3. CONTEXT AND INTEGRATION OF GENERATIVE AI INTO THE LEARNING PROCESS

The case study presented above is particularly well-suited for final-year projects, which offer sufficient time and flexibility for students to engage meaningfully with digital competencies. Nevertheless, the incorporation of such activities into regular coursework may be constrained by limited instructional time and the demands of a rigorous curriculum. In these cases, VBA can be strategically employed to address targeted, well-defined tasks within simplified classroom exercises.

During the 2024–25 academic year, a VBA-based methodology was successfully implemented with third-year undergraduate students enrolled in the Bachelor's Degree in Industrial Chemical Engineering at the University of Huelva (Spain). The experience took place within the Mass Transfer Separations course, a core subject taught by the author (in the summer semester) for over two decades. Instructional activities were conducted using Microsoft Excel spreadsheets, with students required to bring their personal laptops to class. Notably, none of the students had prior experience using VBA macros, and fewer than 10% were even familiar with the concept. This context reflects a common gap in Chemical Engineering education. While students often receive introductory programming instruction (typically in C++), it is rarely connected to their disciplinary applications.

The integration followed a structured framework that can be replicated or adapted by other educators aiming to incorporate programming into technical coursework. The approach consists of the following steps: a) Define a problem involving repetitive calculation tasks to highlight the potential for automation; b) Ask students to solve the problem manually to develop a conceptual understanding and recognize the value of computational tools; c) Introduce VBA incrementally, starting with a basic macro linked to a button for intuitive interaction and ease of use; d) Incorporate generative AI support to assist in generating VBA code from problem descriptions, encouraging students to ask targeted, context-specific questions; e) Assign homework that allows students to independently apply and extend their knowledge using similar problem structures; f) Encourage students to write their own code, using classroom-developed scripts as reference and relying on AI assistance primarily for debugging and refinement.

A representative example involves calculating both the composition profile at the gas–liquid interface and the column height in a packed absorption system. The corresponding Excel-based exercise is provided as Supporting Information (SI_6). For clarity and conciseness, the governing equations and procedural details are outlined separately in Supporting Information (SI_1e).

To determine concentration profiles along a packed absorption column, students must solve a system of equations formed by the Mass Transfer Line and the Equilibrium Line. This is achieved by using Excel's Goal Seek functionality to set the objective function (Mass Transfer Line – Equilibrium Line)

equal to zero. To compute the full profile, the column is discretized into an arbitrary number of segments, and the procedure is repeated at discrete locations from the gas inlet (bottom) to the gas outlet (top).

Initially, students perform this process manually using Goal Seek for each point. This repetitive task is generally perceived as tedious and met with limited enthusiasm. To address this and demonstrate the value of automation in engineering workflows, the instructor introduces a structured methodology for implementing VBA macros, supported by generative AI (GenAI) tools such as Copilot.

The instructional strategy is scaffolded across four progressive stages, each designed to build student confidence and computational fluency:

(i) Macro recording and button integration (SI_6a): Students begin by recording a macro while manually applying Goal Seek to a single data point. This introduces them to the structure of VBA code and its connection to Excel actions. The instructor then demonstrates how to link the macro to a command button, enabling instant execution with a single click. This step emphasizes the power of user interface elements in streamlining calculations.

(ii) Manual replication across multiple points (SI_6b): Next, students are tasked with replicating the macro manually for each of the n points along the column. This involves copying and pasting the Goal Seek instruction and modifying the target cell, changing cell, and target value for each location. While repetitive, this step reinforces the logic behind the automation and prepares students for abstraction.

(iii) AI-assisted looping with VBA (SI_6c): To simplify the code and eliminate redundancy, students are introduced to loop structures in VBA. With guidance from the instructor, they use GenAI to formulate a prompt such as “Provide a VBA script that applies the Goal Seek function across a range of target cells, from row 44 to row 53, where the target and changing cells are in columns G and E, respectively, and the target value is zero.” The AI-generated response introduces the use of For, To, and Next loops, which students analyze and implement. This marks a key learning milestone: transitioning from manual repetition to structured automation. Additional refinement, as demonstrated in Figure 2, may be necessary depending on the complexity of the task.

(iv) Dynamic range detection with AI support (SI_6d): Finally, students are challenged to generalize the script so it can adapt to any number of segments without hardcoding the range. The instructor encourages them to ask: “Based on the previous request, provide a code that automatically detects the number of points beginning from row 44.” The AI returns a script which identifies the last cell with data, allowing the macro to dynamically adjust to the data set. This final step consolidates their understanding of VBA logic, data structures, and the practical use of GenAI as a coding assistant.

With the lecturer's support, a 90 min session is sufficient to complete the entire exercise. After three or four sessions, students are well-equipped to tackle more complex tasks independently. Hence, to further enhance their learning experience and support autonomy, students are then assigned comparable scenarios and encouraged to work independently. Such scenarios are designed to help students reinforce and, mainly, extend what they have learned in classroom exercises (see Supporting Information SI_7). As feedback, the students unanimously remark that writing VBA code in Excel was significantly more practical than the introductory programming

course they had taken in their first year, and that learning VBA offered tangible short-term benefits, particularly in automating the more complex calculations required for their capstone projects.

4. CONCLUDING REMARKS

Excel's logical operators and macros enabled the transformation of traditional mass transfer separation calculations into a functional simulation tool. The resulting interface facilitated clear visualization of process parameters and yielded results consistent with professional software like Aspen Plus. AI assistants, such as Copilot, supported code development and debugging through iterative refinement. As students progress, they gain foundational programming skills and a deeper understanding of process automation. This approach is expected to foster critical thinking and creativity, highlighting the value of integrating computational tools into Chemical Engineering education.

■ ASSOCIATED CONTENT

■ Supporting Information

The Supporting Information is available at <https://pubs.acs.org/doi/10.1021/acs.jchemed.5c01531>.

Procedure and equations to implement the step-by-step method; further information on the software interface; sequence of attempts that improved the "N" macro performance; case study corresponding to the multistage distillation of a ternary mixture; procedure and equations of the absorption classroom exercise (PDF, DOCX)

Video tutorial explaining the processes of recording the "calcTD" macro and creating the "calc T" button (MP4)

Video tutorial showing the creation of the "calcT" macro (MP4)

Excel file containing the absorption classroom exercise; Excel file containing the fractional distillation simulator (ZIP)

Video tutorial detailing how the "N" macro was implemented and assigned to the "Set N" button (MP4)

■ AUTHOR INFORMATION

Corresponding Author

Moisés García-Morales – Departamento de Ingeniería Química, Química Física y Ciencia de los Materiales, Escuela Técnica Superior de Ingeniería, Universidad de Huelva, 21071 Huelva, Spain; Centro de Investigación en Tecnología de Productos y Procesos Químicos (Pro2TecS), Escuela Técnica Superior de Ingeniería, Universidad de Huelva, 21071 Huelva, Spain; orcid.org/0000-0003-4153-487X; Email: moises.garcia@diq.uhu.es

Complete contact information is available at: <https://pubs.acs.org/doi/10.1021/acs.jchemed.5c01531>

Author Contributions

M. García Morales (Conceptualization; Data curation; Formal analysis; Investigation; Methodology; Software; Validation; Visualization; Writing – original draft; Writing – review and editing).

Notes

Declaration of generative AI and AI-assisted technologies in the writing process: During the preparation of this work the author used MS Copilot to improve language and readability of the manuscript. After using this tool/service, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

The author declares no competing financial interest.

Compliance Statement: The research was conducted in accordance with the established ethical standards and adhered to the guidelines set forth by the Journal of Chemical Education for studies involving human subjects.

■ REFERENCES

- (1) Al-Malah, K. I. M. *Aspen Plus, Chemical Engineering Applications*; Wiley & Sons, New Jersey, 2017.
- (2) Haydary, J. *Chemical Process Design and Simulation: Aspen Plus and Aspen Hysys Applications*; Wiley (AIChE), New Jersey, 2019.
- (3) Calvo, L.; Prieto, C. The teaching of enhanced distillation processes using a commercial simulator and a project-based learning approach. *Educ. Chem. Eng.* **2016**, *17*, 65–74.
- (4) Castellón, T.; Botía, D. C.; Gómez, R.; Orozco, G.; Gil, I. D. Using process simulators in the study, design, and control of distillation columns for undergraduate chemical engineering courses. *Comput. Appl. Eng. Educ.* **2011**, *19* (3), 621–630.
- (5) Wankat, P. C. Integrating the use of commercial simulators into lecture courses. *J. Eng. Educ.* **2002**, *91* (1), 19–23.
- (6) Wankat, P. C. Using a commercial simulator to teach sorption separations. *Chem. Eng. Educ.* **2006**, *40* (3), 165–172 <https://journals.flvc.org/cee/article/view/122511>.
- (7) Burns, M. A.; Sung, J. C. Design of separation units using spreadsheets. *Chem. Eng. Educ.* **1996**, *30* (1), 62–69 <https://journals.flvc.org/cee/article/view/123504>.
- (8) Hinestroza, J. P.; Papadopoulos, K. Using spreadsheets and visual basic applications as teaching aids for a unit operations course. *Chem. Eng. Educ.* **2003**, *37* (4), 316–320 <https://journals.flvc.org/cee/article/view/122721>.
- (9) Roman, C.; Delgado, M. A.; García-Morales, M. Using process simulators in Chemical Engineering education: Is it possible to minimize the "black box" effect? *Comput. Appl. Eng. Educ.* **2020**, *28* (5), 1369–1385.
- (10) Briones, L.; Escola, J. M. Application of the Microsoft Excel Solver tool in the solution of optimization problems of heat exchanger network systems. *Educ. Chem. Eng.* **2019**, *26*, 41–47.
- (11) Briones, L.; Morales, V.; Iglesias, J.; Morales, G.; Escola, J. M. Application of the Microsoft Excel Solver tool in the optimization of distillation sequences problems. *Comput. Appl. Eng. Educ.* **2020**, *28* (2), 304–313.
- (12) Clarke, M. A.; Giraldo, C. Using a readily available commercial spreadsheet to teach a graduate course on chemical process simulation. *Chem. Eng. Educ.* **2009**, *43* (2), 160–166 <https://journals.flvc.org/cee/article/view/122334>.
- (13) Ferreira, E. C.; Lima, R.; Salcedo, R. Spreadsheets in Chemical Engineering Education. A Tool in Process Design and Process Integration. *Int. J. Eng. Educ.* **2004**, *20* (6), 928–938 <https://www.ijee.ie/articles/Vol20-6/IJEE1562.pdf>.
- (14) Nachtigalova, I.; Finkeova, J.; Krbcova, Z.; Souskova, H. A spreadsheet-based tool for education of chemical process simulation and control fundamentals. *Comput. Appl. Eng. Educ.* **2020**, *28* (4), 923–937.
- (15) Roman, C.; Delgado, M. A.; García-Morales, M. Bridging the Gap between Hand Calculations and Chemical Process Simulators using Spreadsheets. *Chem. Eng. Educ.* **2021**, *55* (4), 218–225.
- (16) Iglesias, O. A.; Paniagua, C. N. Use of Spreadsheets in Optimization Problems Involving Iterations. *Comput. Appl. Eng. Educ.* **1999**, *7* (4), 227–234.

- (17) Ferreira, E. C.; Salcedo, R. Can Spreadsheet Solvers Solve Demanding Optimization Problems? *Comput. Appl. Eng. Educ.* **2001**, *9*, 49–56.
- (18) Coronell, D. G. Computer Science or Spreadsheet Engineering? An Excel/VBA-Based Programming and Problem Solving Course. *Chem. Eng. Educ.* **2005**, *39* (2), 142–145 <https://journals.flvc.org/cee/article/view/122602>.
- (19) Wu, M.; Di Caprio, U.; Vermeire, F.; Hellinckx, P.; Braeken, L.; Waldherr, S.; Leblebici, M. E. An artificial intelligence course for chemical engineers. *Educ. Chem. Eng.* **2023**, *45*, 141–150.
- (20) Kong, Z. Y.; Kurnia Adi, V. S.; Segovia-Hernández, J. G.; Sunarso, J. Complementary role of large language models in educating undergraduate design of distillation column: Methodology development. *Digit. Chem. Eng.* **2023**, *9*, 100126.
- (21) Caccavale, F.; Gargalo, C. L.; Gernaey, K. V.; Krühne, U. Towards Education 4.0: The role of Large Language Models as virtual tutors in chemical engineering. *Educ. Chem. Eng.* **2024**, *49*, 1–11.
- (22) Gómez-Siurana, A.; Font-Escamilla, A. Spreadsheet to illustrate the application of relaxation methods for multicomponent distillation separations. *Educ. Chem. Eng.* **2023**, *45*, 161–172.
- (23) Kanyarusoke, K. E.; Uziak, J. Spreadsheets: The Ideal Tool for Distance Learning in Engineering Education. *Comput. Appl. Eng. Educ.* **2011**, *19* (2), 216–223.
- (24) Venkatasubramanian, V. Teaching Artificial Intelligence to Chemical Engineers: Experience from A 35-Year-Old Course. *Chem. Eng. Educ.* **2022**, *56* (4), 231–240.
- (25) Wankat, P. C. *Separation Process Engineering, Includes Mass Transfer Analysis*, 4th ed.; Prentice Hall, New York, NY, 2017.
- (26) Gómez-Siurana, A.; Font-Escamilla, A. Novel Spreadsheets to Enhance Learning of Rigorous Equilibrium based Methods for Multicomponent Separations. *Chem. Eng. Educ.* **2022**, *56* (2), 125–134.
- (27) Cardoso, J. P. M.; Tannous, K. Development of a computational tool for designing multicomponent distillation columns. *Comput. Appl. Eng. Educ.* **2020**, *28* (4), 908–922.
- (28) Seader, J. D.; Henley, E. J.; Roper, D. K. *Separation process principles with applications using process simulators*, 4th ed.; Wiley, Hoboken, NJ, 2016.
- (29) Park, J.; Matejka, E.; Nelson, A. K.; Rhodes, J. A. Spreadsheet-based Simulator of a Flash Drum: Class Project Example for Material & Energy Balances and Phase Equilibria Courses. *Chem. Eng. Educ.* **2018**, *52* (4), 251–258.
- (30) Gasteiger, J. Chemistry in Times of Artificial Intelligence. *ChemPhysChem* **2020**, *21*, 2233–2242.
- (31) Hocky, G. M.; White, A. D. Natural language processing models that automate programming will transform chemistry research and teaching. *Digit. Discovery* **2022**, *1*, 79.
- (32) Leite, B. S. Generative Artificial Intelligence in chemistry teaching: ChatGPT, Gemini, and Copilot's content responses. *J. Appl. Learn. Teach.* **2024**, *7* (2), 1–15.
- (33) Wong, K. W. W.; Barford, J. P. Teaching Excel VBA as a problem solving tool for chemical engineering core courses. *Educ. Chem. Eng.* **2010**, *5*, e72–e77.